What Have I Learned from Forth Haiku Thus Far?

2026-03-10

Key Learnings (1/3)

- 2-stage compilation
- Basic shader code does **not** carry state (pure function per pixel)
- Need **2 buffers** and swap between them to access previous state
- Texture memory in GPU can be used to implement state buffers

---

Key Learnings (2/3)

- **Game of Life** is a cellular automaton requiring "state" → needs more than what raw Forth Haiku provides
- `renderer_*.html` as slave can support built-in objects and execute **GLSL**

Key Learnings (3/3)

- **Claude** can build `viewer.html`
  - I'm ignorant of the issues
  - Claude greatly lessens the learning curve
  - I can study the generated code and learn

Variation

- build a pipeline that runs on the command line
- final stage in the pipeline opens a websocket and sends code to browser
- browser runs HTML for CPU renderer or GPU renderer
- websocket daemon and HTML viewer can be tested from command line
- command line == REPL
  - editor modifiest `test.forth`
  - watcher notices update and sends the file through the pipeline

CPU Pipeline

```
file_watcher
  -> compiler
    -> optimizer
      -> boiler plate (wrap_function.js)
        -> websocket daemon (ws_sender.js)
        -> [Javascript]
          -> browser running (CPU) viewer.html
```

CPU Pipeline — Components

- **file_watcher** — polls file for changes (or reacts to system file-changed events)
- **file_watcher** appends "`\\ —EOF—`" to every file send *(Forth Haiku comment syntax)*
- compiler / optimizer / wrapper use "`// —EOF—`" *(Javascript syntax)*
- **websocket daemon** — started before pipeline; opens websocket and keeps it open

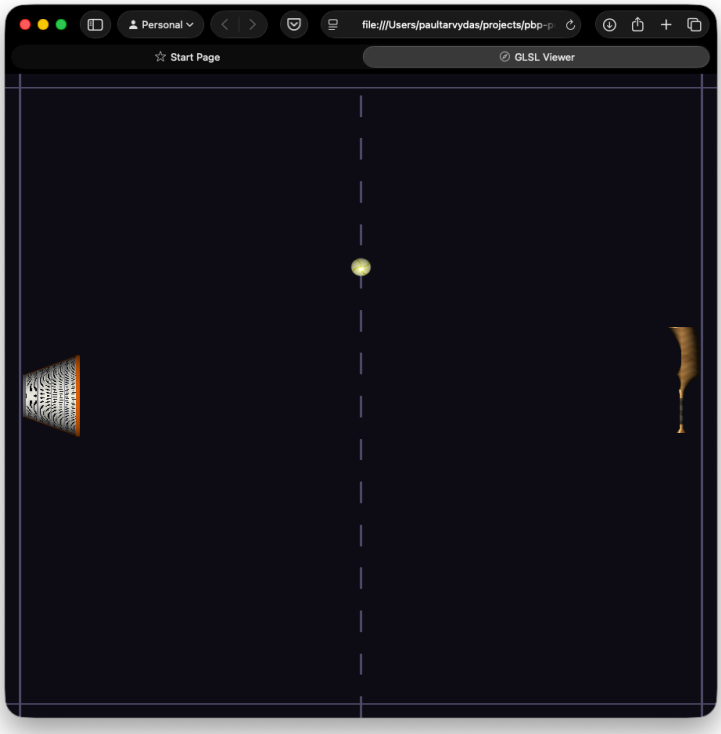
---

GPU Pipeline

```
file_watcher
  -> compiler
    -> optimizer
      -> convert to fragment shader
        -> websocket daemon (ws_sender.js)
        -> [GLSL]
          -> browser running (GLSL) viewer.html
```

Using These Ideas In Other Projects

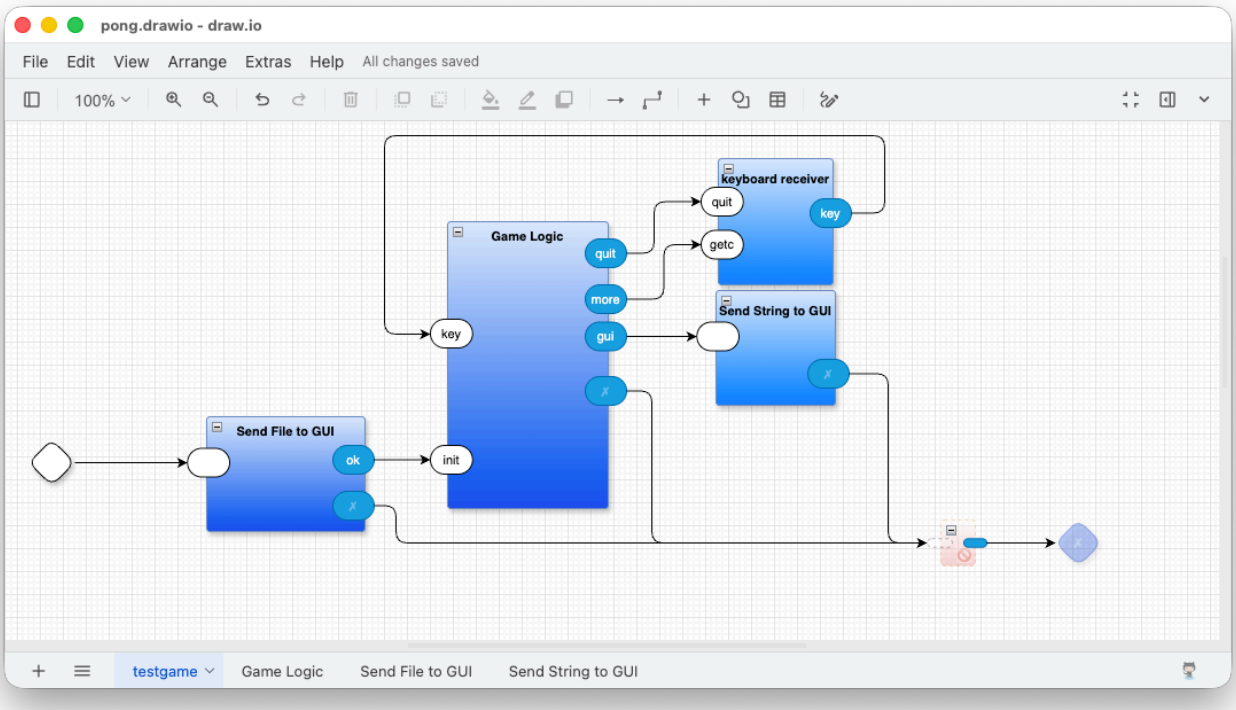
- Command line can send **key-by-key** commands to viewer
- using this to develop and test Pong



- basket, ball, bat, created in browser (renderer_gpu.html)
- keystrokes collected at command line to control position(s) of basket, ball, bat

---

(actual logic not shown in this Forth Haiku repo / example, WIP in Pong repo)



2-Stage Compilation

Why Is This Important?

- simple to do
- opens new doors for approaches to creating machine code
  - like building custom notations, instead of using GPLs

Stage 1 — "Dumb" Code Generation

Each Forth **word** is compiled by:

1. Looking up the word in a **dictionary**
2. Appending the associated text *(list of JS lines using a JS stack)* to the generated code buffer

---

Stage 1 Dictionary

```
function core_words() {
  const dict = {};

  // Stack access
  dict['x'] = ['dstack.push(xpos);'];
  dict['y'] = ['dstack.push(ypos);'];
...
  dict['swap'] = ['work1 = dstack.pop();',
                  'work2 = dstack.pop();',
                  'dstack.push(work1);',
                  'dstack.push(work2);'];
...
```

`x .5 - y .5 - i i i log over`

- x --> `dict['x'] = ['dstack.push(xpos);'];`
- .5 --> `dstack.push(0.5);`
-
    - --> `dict['-'] = ['work1 = dstack.pop();', 'dstack.push(dstack.pop() - work1);'];`
- ...

```
[
  'dstack.push(xpos);',
  'dstack.push(0.5);,
  'work1 = dstack.pop();',
  'dstack.push(dstack.pop() - work1);',
  ...
]
```

Stage 2 — Optimization

Generated code is examined **line by line** using **REGEX**:

- Matches `.push` and `.pop` patterns

- **Pushes** → converted to temporary JS variables, memo'd onto an internal *"simulated"* stack
  - e.g. `temp1`, `temp2`, …
- **Pops** → converted to variable names popped from the simulated stack

---

Appendix

Source Code in Forth

```
\ Primrose haiku
: i 2dup z* log ;
x .5 - y .5 - i i i log over
```

Compiler (Pass 1)

"as is" (machine readable but not formatted for human readability)


```
["dstack.push(xpos);","dstack.push(0.5);","work1 =
dstack.pop();","dstack.push(dstack.pop() -
work1);","dstack.push(ypos);","dstack.push(0.5);","work1 =
dstack.pop();","dstack.push(dstack.pop() - work1);","work1 =
dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 = dstack.pop();","work3 =
dstack.pop();","work4 = dstack.pop();","dstack.push(work4 * work2 - work3 *
work1);","dstack.push(work4 * work1 + work3 *
work2);","dstack.push(Math.log(Math.abs(dstack.pop())));","work1 =
dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 = dstack.pop();","work3 =
dstack.pop();","work4 = dstack.pop();","dstack.push(work4 * work2 - work3 *
work1);","dstack.push(work4 * work1 + work3 *
work2);","dstack.push(Math.log(Math.abs(dstack.pop())));","work1 =
dstack.pop();","work2 =
```

```
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);","work1 = dstack.pop();","work2 = dstack.pop();","work3 =
dstack.pop();","work4 = dstack.pop();","dstack.push(work4 * work2 - work3 *
work1);","dstack.push(work4 * work1 + work3 *
work2);","dstack.push(Math.log(Math.abs(dstack.pop())));","dstack.push(Math.
log(Math.abs(dstack.pop())));","work1 = dstack.pop();","work2 =
dstack.pop();","dstack.push(work2);","dstack.push(work1);","dstack.push(work
2);"]
// ---EOF---
```

Pretty-printed

```
[
    "dstack.push(xpos);",
    "dstack.push(0.5);",
    "work1 = dstack.pop();",
    "dstack.push(dstack.pop() - work1);",
    "dstack.push(ypos);",
    "dstack.push(0.5);",
    "work1 = dstack.pop();",
    "dstack.push(dstack.pop() - work1);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "work3 = dstack.pop();",
    "work4 = dstack.pop();",
    "dstack.push(work4 * work2 - work3 * work1);",
    "dstack.push(work4 * work1 + work3 * work2);",
    "dstack.push(Math.log(Math.abs(dstack.pop())));",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
```

```
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "work3 = dstack.pop();",
    "work4 = dstack.pop();",
    "dstack.push(work4 * work2 - work3 * work1);",
    "dstack.push(work4 * work1 + work3 * work2);",
    "dstack.push(Math.log(Math.abs(dstack.pop())));",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "work3 = dstack.pop();",
    "work4 = dstack.pop();",
    "dstack.push(work4 * work2 - work3 * work1);",
    "dstack.push(work4 * work1 + work3 * work2);",
    "dstack.push(Math.log(Math.abs(dstack.pop())));",
    "dstack.push(Math.log(Math.abs(dstack.pop())));",
    "work1 = dstack.pop();",
    "work2 = dstack.pop();",
    "dstack.push(work2);",
    "dstack.push(work1);",
    "dstack.push(work2);"
]
// ---EOF---
```

Optimizer (Pass 2)

```
[
    "var temp1 = xpos;",
    "var temp2 = 0.5;",
    "work1 = temp2;",
    "var temp3 = temp1 - work1;",
    "var temp4 = ypos;",
    "var temp5 = 0.5;",
    "work1 = temp5;",
    "var temp6 = temp4 - work1;",
    "work1 = temp6;",
    "work2 = temp3;",
    "var temp7 = work2;",
    "var temp8 = work1;",
    "var temp9 = work2;",
    "work1 = temp9;",
    "work2 = temp8;",
    "var temp10 = work2;",
    "var temp11 = work1;",
    "var temp12 = work2;",
    "work1 = temp12;",
    "work2 = temp11;",
    "work3 = temp10;",
    "work4 = temp7;",
    "var temp13 = work4 * work2 - work3 * work1;",
    "var temp14 = work4 * work1 + work3 * work2;",
    "var temp15 = Math.log(Math.abs(temp14));",
    "work1 = temp15;",
    "work2 = temp13;",
    "var temp16 = work2;",
    "var temp17 = work1;",
    "var temp18 = work2;",
    "work1 = temp18;",
    "work2 = temp17;",
    "var temp19 = work2;",
    "var temp20 = work1;",
    "var temp21 = work2;",
    "work1 = temp21;",
    "work2 = temp20;",
    "work3 = temp19;",
    "work4 = temp16;",
    "var temp22 = work4 * work2 - work3 * work1;",
    "var temp23 = work4 * work1 + work3 * work2;",
    "var temp24 = Math.log(Math.abs(temp23));",
    "work1 = temp24;",
    "work2 = temp22;",
    "var temp25 = work2;",
```

```
    "var temp26 = work1;",
    "var temp27 = work2;",
    "work1 = temp27;",
    "work2 = temp26;",
    "var temp28 = work2;",
    "var temp29 = work1;",
    "var temp30 = work2;",
    "work1 = temp30;",
    "work2 = temp29;",
    "work3 = temp28;",
    "work4 = temp25;",
    "var temp31 = work4 * work2 - work3 * work1;",
    "var temp32 = work4 * work1 + work3 * work2;",
    "var temp33 = Math.log(Math.abs(temp32));",
    "var temp34 = Math.log(Math.abs(temp33));",
    "work1 = temp34;",
    "work2 = temp31;",
    "var temp35 = work2;",
    "var temp36 = work1;",
    "var temp37 = work2;",
    "dstack.push(temp35);",
    "dstack.push(temp36);",
    "dstack.push(temp37);",
    "dstack.push(1.0);"
]
// ---EOF---
```

Wrap Function

```
var go = function(
    time_val, time_delta_val,
    xpos, ypos,
    mouse_x, mouse_y,
    button_val,
    memory) {
    var PI = Math.PI;
    var random = Math.random;
    var floor = Math.floor;
    var ceil = Math.ceil;
    var min = Math.min;
    var max = Math.max;
    var log = Math.log;
    var sqrt = Math.sqrt;
```

```
var pow = Math.pow;
var abs = Math.abs;
var sin = Math.sin;
var cos = Math.cos;
var tan = Math.tan;
var atan2 = Math.atan2;
var exp = Math.exp;
var audio_sample = 0.0;

function hasbit(val, b) {
    b = Math.floor(b);
    return mod(val, Math.pow(2.0, b + 1)) >= Math.pow(2.0, b);
}
function sample(x, y) {}
function sample_r() { return 0.0; }
function sample_g() { return 0.9; }
function sample_b() { return 0.7; }
function store(v, addr) {
    memory[mod(Math.floor(addr), 16)] = v;
}
function load(addr) {
    return memory[mod(Math.floor(addr), 16)];
}
function mod(v1, v2) {
    return v1 - v2 * Math.floor(v1 / v2);
}

var dstack = [];
var rstack = [];
var work1, work2, work3, work4;

var temp1 = xpos;
var temp2 = 0.5;
work1 = temp2;
var temp3 = temp1 - work1;
var temp4 = ypos;
var temp5 = 0.5;
work1 = temp5;
var temp6 = temp4 - work1;
work1 = temp6;
work2 = temp3;
var temp7 = work2;
var temp8 = work1;
var temp9 = work2;
work1 = temp9;
work2 = temp8;
```

```
    var temp10 = work2;
    var temp11 = work1;
    var temp12 = work2;
    work1 = temp12;
    work2 = temp11;
    work3 = temp10;
    work4 = temp7;
    var temp13 = work4 * work2 - work3 * work1;
    var temp14 = work4 * work1 + work3 * work2;
    var temp15 = Math.log(Math.abs(temp14));
    work1 = temp15;
    work2 = temp13;
    var temp16 = work2;
    var temp17 = work1;
    var temp18 = work2;
    work1 = temp18;
    work2 = temp17;
    var temp19 = work2;
    var temp20 = work1;
    var temp21 = work2;
    work1 = temp21;
    work2 = temp20;
    work3 = temp19;
    work4 = temp16;
    var temp22 = work4 * work2 - work3 * work1;
    var temp23 = work4 * work1 + work3 * work2;
    var temp24 = Math.log(Math.abs(temp23));
    work1 = temp24;
    work2 = temp22;
    var temp25 = work2;
    var temp26 = work1;
    var temp27 = work2;
    work1 = temp27;
    work2 = temp26;
    var temp28 = work2;
    var temp29 = work1;
    var temp30 = work2;
    work1 = temp30;
    work2 = temp29;
    work3 = temp28;
    work4 = temp25;
    var temp31 = work4 * work2 - work3 * work1;
    var temp32 = work4 * work1 + work3 * work2;
    var temp33 = Math.log(Math.abs(temp32));
    var temp34 = Math.log(Math.abs(temp33));
    work1 = temp34;
```

```
    work2 = temp31;
    var temp35 = work2;
    var temp36 = work1;
    var temp37 = work2;
    dstack.push(temp35);
    dstack.push(temp36);
    dstack.push(temp37);
    dstack.push(1.0);

    return dstack;
};
go
// ---EOF---
```

CPU Viewer

renderer_cpu.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Forth Haiku Renderer</title>
    <style>
      body {
          margin: 0;
          padding: 20px;
          font-family: monospace;
          background: #222;
          color: #eee;
      }
      #container {
          display: flex;
          gap: 20px;
          align-items: flex-start;
      }
      canvas {
          border: 2px solid #666;
          background: black;
          image-rendering: pixelated;
      }
      #info {
          font-size: 14px;
```

```
      }
      .status {
          padding: 10px;
          margin: 10px 0;
          border-radius: 4px;
      }
      .connected { background: #0a4; }
      .disconnected { background: #a40; }
      .error { background: #a00; }
      #code {
          background: #333;
          padding: 10px;
          border-radius: 4px;
          max-height: 400px;
          overflow-y: auto;
          font-size: 12px;
          white-space: pre;
      }
    </style>
  </head>
  <body>
    <h1>Forth Haiku Live Renderer</h1>

    <div id="container">
      <div>
        <canvas id="canvas" width="512" height="512"></canvas>
        <div style="margin-top: 10px;">
          <label>
            <input type="checkbox" id="animate" checked> Animate
          </label>
          <span style="margin-left: 20px;">FPS: <span id="fps">0</span>
</span>
        </div>
      </div>

      <div id="info">
        <div id="status" class="status disconnected">Connecting to
WebSocket...</div>
        <div style="margin-top: 20px;">
          <strong>Received Code:</strong>
          <div id="code">No code yet...</div>
        </div>
      </div>
    </div>

    <script>
```

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');
const statusEl = document.getElementById('status');
const codeEl = document.getElementById('code');
const fpsEl = document.getElementById('fps');
const animateCheckbox = document.getElementById('animate');

const w = canvas.width;
const h = canvas.height;
const imageData = ctx.createImageData(w, h);
const memory = new Float32Array(16);

let currentFunc = null;
let startTime = Date.now();
let lastTime = 0;
let frameCount = 0;
let lastFpsUpdate = Date.now();

// Mouse tracking
let mouseX = 0;
let mouseY = 0;
let mouseButtons = 0;

canvas.addEventListener('mousemove', (e) => {
    const rect = canvas.getBoundingClientRect();
    mouseX = (e.clientX - rect.left) / rect.width;
    mouseY = (e.clientY - rect.top) / rect.height;
});

canvas.addEventListener('mousedown', (e) => {
    mouseButtons |= 1;
});

canvas.addEventListener('mouseup', (e) => {
    mouseButtons &= ~1;
});

// Connect to WebSocket
const ws = new WebSocket('ws://localhost:8080');

ws.onopen = () => {
    console.log('Connected to WebSocket server');
    statusEl.className = 'status connected';
    statusEl.textContent = 'Connected to WebSocket server';
};
```

```
        ws.onclose = () => {
            console.log('Disconnected from WebSocket server');
            statusEl.className = 'status disconnected';
            statusEl.textContent = 'Disconnected from WebSocket server';
        };

        ws.onerror = (error) => {
            console.error('WebSocket error:', error);
            statusEl.className = 'status error';
            statusEl.textContent = 'WebSocket error - check console';
        };

        ws.onmessage = (event) => {
            const code = event.data;
            console.log('Received code:', code.substring(0, 100) + '...');

            // Display code
            codeEl.textContent = code;

            try {
                // Eval the code to create the function
                currentFunc = eval(code);

                statusEl.className = 'status connected';
                statusEl.textContent = 'Code received and compiled
successfully!';

                // Reset time
                startTime = Date.now();
                lastTime = 0;

                console.log('Function compiled successfully');
            } catch (e) {
                console.error('Error compiling code:', e);
                statusEl.className = 'status error';
                statusEl.textContent = 'Compilation error: ' + e.message;
            }
        };

        function render() {
            if (!currentFunc) {
                requestAnimationFrame(render);
                return;
            }

            const now = Date.now();
```

```
            const time = (now - startTime) / 1000.0;
            const dt = time - lastTime;
            lastTime = time;

            // Render each pixel
            for (let y = 0; y < h; y++) {
                for (let x = 0; x < w; x++) {
                    try {
                        const result = currentFunc(
                            time,           // time_val
                            dt,             // time_delta_val
                            x / w,          // xpos (0..1)
                            y / h,          // ypos (0..1)
                            mouseX,         // mouse_x
                            mouseY,         // mouse_y
                            mouseButtons,   // button_val
                            memory          // memory array
                        );

                        const idx = (y * w + x) * 4;
                        imageData.data[idx + 0] = Math.floor(result[0] * 255);
// R
                        imageData.data[idx + 1] = Math.floor(result[1] * 255);
// G
                        imageData.data[idx + 2] = Math.floor(result[2] * 255);
// B
                        imageData.data[idx + 3] = Math.floor(result[3] * 255);
// A
                    } catch (e) {
                        // If function throws, use error color (red)
                        const idx = (y * w + x) * 4;
                        imageData.data[idx + 0] = 255;
                        imageData.data[idx + 1] = 0;
                        imageData.data[idx + 2] = 0;
                        imageData.data[idx + 3] = 255;
                    }
                }
            }

            ctx.putImageData(imageData, 0, 0);

            // Update FPS
            frameCount++;
            if (now - lastFpsUpdate > 1000) {
                fpsEl.textContent = frameCount.toFixed(1);
                frameCount = 0;
```

```
                lastFpsUpdate = now;
            }

            if (animateCheckbox.checked) {
                requestAnimationFrame(render);
            }
        }

        // Start animation loop
        render();

        // Re-render when animate checkbox changes
        animateCheckbox.addEventListener('change', () => {
            if (animateCheckbox.checked) {
                render();
            }
        });
    </script>
  </body>
</html>
```

Example Code (GPU pipeline)

Source Code in Forth

(same)

Compiler (Pass 1)

(same)

Optimizer (Pass 2)

(same)

Fragment Shader

```
precision highp float;
varying vec2 tpos;
uniform float time_val, time_delta_val, button_val, mouse_x, mouse_y;
uniform float memory_val[16];
float memory[16]; float audio_sample;
float gsin(float v) { return sin(mod(v, 6.283185307)); }
float gcos(float v) { return cos(mod(v, 6.283185307)); }
float gtan(float v) { return tan(mod(v, 6.283185307)); }
float hasbit(float v, float b) { b = floor(b); return mod(v, pow(2.0, b +
1.0)) >= pow(2.0, b) ? 1.0 : 0.0; }
float load(float a) { int ai = int(mod(floor(a), 16.0)); for (int i = 0; i <
16; ++i) { if (i == ai) return memory[i]; } return 0.0; }
void store(float v, float a) { int ai = int(mod(floor(a), 16.0)); for (int i
= 0; i < 16; ++i) { if (i == ai) memory[i] = v; } }
void main(void) {
    float work1, work2, work3, work4, seed;
    for (int i = 0; i < 16; ++i) { memory[i] = memory_val[i]; }
    float temp1 = tpos.x;
    float temp2 = 0.5;
    work1 = temp2;
    float temp3 = temp1 - work1;
    float temp4 = tpos.y;
    float temp5 = 0.5;
    work1 = temp5;
    float temp6 = temp4 - work1;
    work1 = temp6;
    work2 = temp3;
    float temp7 = work2;
    float temp8 = work1;
    float temp9 = work2;
    work1 = temp9;
    work2 = temp8;
    float temp10 = work2;
    float temp11 = work1;
    float temp12 = work2;
    work1 = temp12;
    work2 = temp11;
    work3 = temp10;
    work4 = temp7;
    float temp13 = work4 * work2 - work3 * work1;
    float temp14 = work4 * work1 + work3 * work2;
    float temp15 = log(abs(temp14));
    work1 = temp15;
```

```
    work2 = temp13;
    float temp16 = work2;
    float temp17 = work1;
    float temp18 = work2;
    work1 = temp18;
    work2 = temp17;
    float temp19 = work2;
    float temp20 = work1;
    float temp21 = work2;
    work1 = temp21;
    work2 = temp20;
    work3 = temp19;
    work4 = temp16;
    float temp22 = work4 * work2 - work3 * work1;
    float temp23 = work4 * work1 + work3 * work2;
    float temp24 = log(abs(temp23));
    work1 = temp24;
    work2 = temp22;
    float temp25 = work2;
    float temp26 = work1;
    float temp27 = work2;
    work1 = temp27;
    work2 = temp26;
    float temp28 = work2;
    float temp29 = work1;
    float temp30 = work2;
    work1 = temp30;
    work2 = temp29;
    work3 = temp28;
    work4 = temp25;
    float temp31 = work4 * work2 - work3 * work1;
    float temp32 = work4 * work1 + work3 * work2;
    float temp33 = log(abs(temp32));
    float temp34 = log(abs(temp33));
    work1 = temp34;
    work2 = temp31;
    float temp35 = work2;
    float temp36 = work1;
    float temp37 = work2;
    gl_FragColor = vec4(temp35, temp36, temp37, 1.0);
    gl_FragColor = clamp(gl_FragColor, 0.0, 1.0);
    gl_FragColor.rgb *= gl_FragColor.a;
}
// ---EOF---
```

GPU Viewer

renderer_gpu.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>GPU Haiku Renderer</title>
  <style>
    body { margin: 0; padding: 20px; font-family: monospace; background:
#222; color: #eee; }
    canvas { border: 2px solid #666; background: black; }
    .status { padding: 10px; margin: 10px 0; border-radius: 4px; }
    .connected { background: #0a4; }
    .error { background: #a00; }
  </style>
</head>
<body>
  <h1>GPU Haiku Renderer</h1>
  <canvas id="canvas" width="512" height="512"></canvas>
  <div id="status" class="status">Connecting...</div>
  <div>FPS: <span id="fps">0</span></div>

  <script>
    const canvas = document.getElementById('canvas');
    const statusEl = document.getElementById('status');
    const fpsEl = document.getElementById('fps');

    let gl, program, startTime = Date.now(), frameCount = 0, lastFps =
Date.now();
    let mouseX = 0, mouseY = 0, mouseButtons = 0;
    const memory = new Float32Array(16);

    canvas.addEventListener('mousemove', (e) => {
      const r = canvas.getBoundingClientRect();
      mouseX = (e.clientX - r.left) / r.width;
      mouseY = 1.0 - (e.clientY - r.top) / r.height;
    });
    canvas.addEventListener('mousedown', () => mouseButtons |= 1);
    canvas.addEventListener('mouseup', () => mouseButtons &= ~1);

    function setupWebGL(glslCode) {
      gl = canvas.getContext('webgl');
      if (!gl) throw new Error('WebGL not supported');
```

```
      const vcode = 'attribute vec2 ppos; varying vec2 tpos; void main() {
tpos = (ppos + 1.0) / 2.0; gl_Position = vec4(ppos, 0.0, 1.0); }';
      const vs = gl.createShader(gl.VERTEX_SHADER);
      gl.shaderSource(vs, vcode);
      gl.compileShader(vs);
      if (!gl.getShaderParameter(vs, gl.COMPILE_STATUS)) throw new
Error('VS: ' + gl.getShaderInfoLog(vs));

      const fs = gl.createShader(gl.FRAGMENT_SHADER);
      gl.shaderSource(fs, glslCode);
      gl.compileShader(fs);
      if (!gl.getShaderParameter(fs, gl.COMPILE_STATUS)) throw new
Error('FS:\n' + gl.getShaderInfoLog(fs));

      program = gl.createProgram();
      gl.attachShader(program, vs);
      gl.attachShader(program, fs);
      gl.linkProgram(program);
      if (!gl.getProgramParameter(program, gl.LINK_STATUS)) throw new
Error('Link: ' + gl.getProgramInfoLog(program));
      gl.useProgram(program);

      const verts = new Float32Array([-1,-1, 1,-1, 1,1, -1,-1, 1,1, -1,1]);
      const vb = gl.createBuffer();
      gl.bindBuffer(gl.ARRAY_BUFFER, vb);
      gl.bufferData(gl.ARRAY_BUFFER, verts, gl.STATIC_DRAW);
      const va = gl.getAttribLocation(program, 'ppos');
      gl.enableVertexAttribArray(va);
      gl.vertexAttribPointer(va, 2, gl.FLOAT, false, 0, 0);
    }

    function render() {
      if (gl && program) {
        const t = (Date.now() - startTime) / 1000;
        gl.uniform1f(gl.getUniformLocation(program, 'time_val'), t);
        gl.uniform1f(gl.getUniformLocation(program, 'time_delta_val'),
1/60);
        gl.uniform1f(gl.getUniformLocation(program, 'mouse_x'), mouseX);
        gl.uniform1f(gl.getUniformLocation(program, 'mouse_y'), mouseY);
        gl.uniform1f(gl.getUniformLocation(program, 'button_val'),
mouseButtons);
        gl.uniform1fv(gl.getUniformLocation(program, 'memory_val'), memory);
        gl.clear(gl.COLOR_BUFFER_BIT);
        gl.drawArrays(gl.TRIANGLES, 0, 6);
        if (Date.now() - lastFps > 1000) { fpsEl.textContent = frameCount;
frameCount = 0; lastFps = Date.now(); }
```

```
        frameCount++;
      }
      requestAnimationFrame(render);
    }

    const ws = new WebSocket('ws://localhost:8080');
    ws.onopen = () => { statusEl.className = 'status connected';
statusEl.textContent = 'Connected'; };
    ws.onclose = () => statusEl.textContent = 'Disconnected';
    ws.onerror = () => { statusEl.className = 'status error';
statusEl.textContent = 'Error'; };
    ws.onmessage = (e) => {
      try {
        const glsl = e.data.replace(/\/\/ ---EOF---[\s\S]*$/, '').trim();
        console.log('Received GLSL (' + glsl.length + ' bytes)');
        setupWebGL(glsl);
        statusEl.textContent = 'Compiled!';
        startTime = Date.now();
      } catch (err) {
        statusEl.className = 'status error';
        statusEl.textContent = 'Error: ' + err.message;
        console.error(err);
      }
    };

    render();
  </script>
</body>
</html>
```

Code Repository

https://github.com/guitarvydas/forth-haiku
